

'''

1. Linear Regression — Full Notes (Beginner → Intermediate)

Linear Regression is a Supervised Machine Learning Algorithm used to predict continuous numerical values.

Examples:

- House Price Prediction
- Salary Prediction
- Temperature Forecast
- Sales Prediction

What is Linear Regression?

Linear Regression finds the linear relationship between input and output.

Example:

Area (x) | Price (y)

1000 | 200

1500 | 300

2000 | 400

As area increases, price increases.

When plotted on a graph, it forms a straight line.

Formula:

$$y = mx + c$$

Where:

- $y \rightarrow$ Predicted value
- $x \rightarrow$ Input feature

- $m \rightarrow$ Slope
- $c \rightarrow$ Intercept

2. Simple Linear Regression

Used when there is only one feature.

Formula:

$$y = mx + c$$

Example:

$$y = 2x + 5$$

If:

$$x = 10$$

Then:

$$y = 2(10) + 5$$

$$y = 25$$

3. Slope Formula

$$m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

Where:

- x_i = Input value
- y_i = Output value
- $\bar{x}$ = Mean of x
- $\bar{y}$ = Mean of y

Given:

$$n = 3$$

$$x = [1000, 1500, 2000]$$

$$y = [200, 300, 400]$$

Step 1: Calculate the means

$$\begin{aligned}\bar{x} &= (1000 + 1500 + 2000) / 3 \\ &= 4500 / 3 \\ &= 1500\end{aligned}$$

$$\begin{aligned}\bar{y} &= (200 + 300 + 400) / 3 \\ &= 900 / 3 \\ &= 300\end{aligned}$$

Step 2: Calculate the required values

xi	yi	xi - $\bar{x}$	yi - $\bar{y}$	(xi - $\bar{x}$)(yi - $\bar{y}$)	(xi - $\bar{x}$) ²
-----	-----	-----	-----	-----	-----
1000	200	-500	-100	50,000	250,000
1500	300	0	0	0	0
2000	400	500	100	50,000	250,000

$$\Sigma(xi - \bar{x})(yi - \bar{y}) = 100,000$$

$$\Sigma(xi - \bar{x})^2 = 500,000$$

Step 3: Calculate the slope

$$m = 100,000 / 500,000$$

$$m = 0.2$$

Final Answer:

$$\text{Slope (m)} = 0.2$$

4. Intercept Formula

$$c = \bar{y} - m\bar{x}$$

$$c = 300 - (0.2 * 1500)$$

$$c = 300 - 300$$

$$c = 0$$

Final Answer:

$$c = 0$$

5. Prediction Formula

$$\hat{y} = mx + c$$

Where:

$\hat{y}$ = Predicted value of y

m = Slope of the regression line

x = Input (independent variable)

c = Intercept (constant)

Given:

$$m = 0.2$$

$$c = 0$$

Therefore,

$$\hat{y} = 0.2x + 0$$

$$\hat{y} = 0.2x$$

Example:

If $x = 2500$,

$$\hat{y} = 0.2 * 2500$$

$$\hat{y} = 500$$

Predicted value = 500

6. Cost Function

Cost Function is used to measure the error between the actual values and the predicted values.

The goal of Linear Regression is to minimize the cost (error).

Mean Squared Error (MSE):

$$J(m,c) = (1/n) \sum (y_i - \hat{y}_i)^2$$

Where:

- $J(m,c)$ = Cost Function (Error)
- n = Number of data points
- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- m = Slope
- c = Intercept

Given:

$$m = 0.2$$

$$c = 0$$

Regression Equation:

$$\hat{y} = 0.2x$$

Data:

x	Actual (y)	Predicted ($\hat{y}$)
1000	200	200
1500	300	300
2000	400	400

Step 1: Calculate Errors

Actual (y)	Predicted ($\hat{y}$)	Error (y - $\hat{y}$)	Squared Error (y - $\hat{y}$) ²
200	200	0	0
300	300	0	0
400	400	0	0

Step 2: Calculate the Cost Function

$$J(0.2,0) = (1/3) [(200 - 200)^2 + (300 - 300)^2 + (400 - 400)^2]$$

$$= (1/3) (0 + 0 + 0)$$

$$= 0/3$$

$$= 0$$

Final Answer:

Cost Function (MSE) = 0

Interpretation:

- MSE = 0 → Perfect prediction (No error)
- Lower MSE → Better model
- Higher MSE → More prediction error

Another Way to Find the Optimal Values of m and c

Why does the number 2 appear in the Partial Derivative?

The number 2 comes from taking the derivative of the squared term in the Cost Function.

Cost Function:

$$J(m,c) = (1/n) \sum (y_i - \hat{y}_i)^2$$

The squared error term is:

$$(y_i - \hat{y}_i)^2$$

Using the Power Rule of differentiation:

$$d(x^2)/dx = 2x$$

Therefore,

$$\begin{aligned} d[(y_i - \hat{y}_i)^2]/dm \\ = 2(y_i - \hat{y}_i) \times d(y_i - \hat{y}_i)/dm \end{aligned}$$

Since

$$\hat{y}_i = mx_i + c$$

Then,

$$\begin{aligned} d(y_i - \hat{y}_i)/dm \\ = d(y_i - mx_i - c)/dm \\ = -x_i \end{aligned}$$

Substituting this into the equation:

$$\partial J / \partial m = -(2/n) \sum x_i (y_i - \hat{y}_i)$$

Similarly,

$$\partial J / \partial c = -(2/n) \sum (y_i - \hat{y}_i)$$

Summary:

- The number 2 comes from the derivative of the squared term (x^2).

- The negative sign (-) comes from differentiating $(y_i - \hat{y}_i)$.
- The term $1/n$ comes from the Cost Function formula.

Example

Given:

$$m = 0.2$$

$$c = 0$$

$$x = [1000, 1500, 2000]$$

$$y = [200, 300, 400]$$

Regression Equation:

$$\hat{y} = mx + c$$

Step 1: Calculate Predicted Values

$$\hat{y} = [200, 300, 400]$$

Step 2: Calculate the Partial Derivative with respect to m

$$\partial J / \partial m = -(2/3)[1000(200-200) + 1500(300-300) + 2000(400-400)]$$

$$= -(2/3)(0 + 0 + 0)$$

$$= 0$$

Step 3: Calculate the Partial Derivative with respect to c

$$\partial J / \partial c = -(2/3)[(200-200) + (300-300) + (400-400)]$$

$$= -(2/3)(0)$$

$$= 0$$

Since

$$\partial J / \partial m = 0$$

and

$$\partial J / \partial c = 0$$

the values

$$m = 0.2$$

$$c = 0$$

minimize the Cost Function.

Step 4: Calculate the Cost Function

$$J = (1/3)[(200-200)^2 + (300-300)^2 + (400-400)^2]$$

$$= 0$$

Final Answer:

$$m = 0.2$$

$$c = 0$$

$$\text{Cost Function (MSE)} = 0$$

F1 Score measures the balance between Precision and Recall.

Formula:

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

It combines Precision and Recall into a single performance metric.

It is the Harmonic Mean of Precision and Recall.

It is useful when both False Positives (FP) and False Negatives (FN) are important.

Range:

- F1 Score = 1 → Perfect classification
- F1 Score = 0 → Poor classification

Higher F1 Score indicates a better classification model.

7. MAE (Mean Absolute Error)

Mean Absolute Error (MAE) measures the average absolute difference between the actual values and the predicted values.

Formula:

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

Where:

- MAE = Mean Absolute Error
- n = Number of data points
- y_i = Actual value
- $\hat{y}_i$ = Predicted value

Example:

Actual Values (y):

[10, 20]

Predicted Values ($\hat{y}$):

[12, 18]

Step 1: Calculate Absolute Errors

$$|10 - 12| = 2$$

$$|20 - 18| = 2$$

Step 2: Calculate MAE

$$\text{MAE} = (2 + 2) / 2$$

$$= 4 / 2$$

$$= 2$$

Final Answer:

$$\text{MAE} = 2$$

Interpretation:

- MAE = 0 → Perfect prediction
- Lower MAE → Better model
- Higher MAE → More prediction error

8. MSE (Mean Squared Error)

Mean Squared Error (MSE) measures the average of the squared differences between the actual values and the predicted values.

Formula:

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Where:

- MSE = Mean Squared Error
- n = Number of data points
- y_i = Actual value
- $\hat{y}_i$ = Predicted value

Example:

Actual Values (y):

[10, 20]

Predicted Values ($\hat{y}$):

[12, 18]

Step 1: Calculate Errors

$$10 - 12 = -2$$

$$20 - 18 = 2$$

Errors:

[-2, 2]

Step 2: Square the Errors

$$(-2)^2 = 4$$

$$(2)^2 = 4$$

Squared Errors:

[4, 4]

Step 3: Calculate MSE

$$\text{MSE} = (4 + 4) / 2$$

$$= 8 / 2$$

$$= 4$$

Final Answer:

$$\text{MSE} = 4$$

Interpretation:

- $\text{MSE} = 0 \rightarrow$ Perfect prediction
- Lower MSE $\rightarrow$ Better model
- Higher MSE $\rightarrow$ More prediction error
- MSE penalizes larger errors more because the errors are squared.

9. RMSE (Root Mean Squared Error)

Root Mean Squared Error (RMSE) is the square root of the Mean Squared Error (MSE).

Formula:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Where:

- RMSE = Root Mean Squared Error
- MSE = Mean Squared Error

Example:

From the previous example,

$$\text{MSE} = 4$$

Step 1: Calculate RMSE

$$\text{RMSE} = \sqrt{4}$$

$$= 2$$

Final Answer:

$$\text{RMSE} = 2$$

Interpretation:

- $\text{RMSE} = 0 \rightarrow$ Perfect prediction
- Lower RMSE $\rightarrow$ Better model
- Higher RMSE $\rightarrow$ More prediction error

Advantages:

- Uses the same unit as the target (actual) variable.
- Easy to interpret.
- Penalizes larger errors more than MAE because the errors are squared before taking the square root.

10. R^2 Score (Coefficient of Determination)

R^2 Score measures how well the Linear Regression model fits the data.

Formula:

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$$

Where:

- R^2 = Coefficient of Determination
- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- $\bar{y}$ = Mean of the actual values

Example:

Actual Values (y):

[10, 20, 30]

Predicted Values ($\hat{y}$):

[12, 18, 28]

Step 1: Calculate Mean of Actual Values

$$\bar{y} = (10 + 20 + 30) / 3$$

$$= 60 / 3$$

$$= 20$$

Step 2: Calculate Sum of Squared Errors (SSE)

$$\Sigma(y_i - \hat{y}_i)^2$$

$$= (10 - 12)^2 + (20 - 18)^2 + (30 - 28)^2$$

$$= 4 + 4 + 4$$

$$= 12$$

Step 3: Calculate Total Sum of Squares (SST)

$$\Sigma(y_i - \bar{y})^2$$

$$= (10 - 20)^2 + (20 - 20)^2 + (30 - 20)^2$$

$$= 100 + 0 + 100$$

$$= 200$$

Step 4: Calculate R² Score

$$R^2 = 1 - (12 / 200)$$

$$= 1 - 0.06$$

$$= 0.94$$

Final Answer:

$$R^2 = 0.94$$

Interpretation:

- $R^2 = 1 \rightarrow$ Perfect fit
- $R^2 \approx 0 \rightarrow$ Model performs no better than predicting the mean
- $R^2 < 0 \rightarrow$ Model performs worse than predicting the mean

Range:

- 1 $\rightarrow$ Perfect model
- 0 $\rightarrow$ Average model
- Less than 0 $\rightarrow$ Poor model

11. Multiple Linear Regression

Used with multiple features.

Multiple Linear Regression Formula

Formula:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Where:

y = Output (Predicted value)

b_0 = Intercept

$b_1, b_2, \dots, b_n$ = Coefficients (weights)

$x_1, x_2, \dots, x_n$ = Input features

Example:

$$\text{Price} = 5000 + 2000(\text{Age}) + 300(\text{Size})$$

Given:

Age = 10 years

Size = 1000 sq ft

Calculation:

$$\text{Price} = 5000 + 2000(10) + 300(1000)$$

$$\text{Price} = 5000 + 20000 + 300000$$

$$\text{Price} = 325000$$

$$\text{Predicted House Price} = 325000$$

12. Vector Form

$$Y = X\beta + \varepsilon$$

Where:

- Y = Target (Output Vector)
- X = Feature Matrix
- β = Coefficient Vector
- ε = Error Vector

Example:

$$Y = \begin{bmatrix} 200 \\ 300 \\ 400 \end{bmatrix}$$

$$X = \begin{bmatrix} 1000 & 1 \\ 1500 & 1 \\ 2000 & 1 \end{bmatrix}$$

$$\beta = \begin{bmatrix} 0.2 \end{bmatrix}$$

$$[0]]$$

$$\varepsilon = [[0],$$

$$[0],$$

$$[0]]$$

Multiple Linear Regression Matrix Formula:

$$Y = X\beta + \varepsilon$$

Since $\varepsilon = 0$,

$$Y = X\beta$$

$$Y = [[1000, 1],$$

$$[1500, 1],$$

$$[2000, 1]]$$

$$\times$$

$$[[0.2],$$

$$[0]]$$

1st row:

$$1000 \times 0.2 + 1 \times 0 = 200$$

2nd row:

$$1500 \times 0.2 + 1 \times 0 = 300$$

3rd row:

$$2000 \times 0.2 + 1 \times 0 = 400$$

Therefore,

$Y = \begin{bmatrix} 200 \\ 300 \\ 400 \end{bmatrix}$

The predicted values are exactly equal to the actual values.

That means:

- Coefficient (Slope) = 0.2
- Intercept = 0
- Error (ϵ) = 0
- Model fits the data perfectly ($R^2 = 1$)

Regression Equation:

$$y = 0.2x$$

Interpretation:

For every 1 unit increase in x , the predicted value of y increases by 0.2 units.

13. Normal Equation

Directly calculates coefficients.

Formula:

$$\beta = (X^T X)^{-1} X^T y$$

Meaning:

- X = Input Matrix

- X^T = Transpose
- $(X^T X)^{-1}$ = Matrix Inverse
- y = Output
- β = Weights

Example:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

$$y = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

Result:

$$\beta = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

Equation:

$$y = 2x$$

14. Gradient Descent

Iterative optimization.

Update:

$$m = m - \alpha(\partial J / \partial m)$$

$$c = c - \alpha(\partial J / \partial c)$$

Where: α = Learning Rate

Large $\alpha \rightarrow$ Overshoot

Small $\alpha \rightarrow$ Slow

15. Assumptions of Linear Regression

- Linearity
- Independence
- Homoscedasticity
- Normality
- No Multicollinearity

16. Train Test Split

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42  
)
```

80% Train

20% Test

17. Build Linear Regression

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
model.fit( X_train, y_train )
```

18. Prediction

```
y_pred = model.predict(X_test)
```

19. Evaluation

```
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score )
```

20. Coefficients

```
print("Coefficients:", model.coef_)  
print("Intercept:", model.intercept_)
```

21. Visualization

```
import seaborn as sns  
import matplotlib.pyplot as plt  
sns.scatterplot( x=y_test, y=y_pred )  
plt.show()
```

22. Advantages

✓ Easy

- ✓ Fast
- ✓ Interpretable
- ✓ Works well for linear data

23. Disadvantages

- ✗ Sensitive to outliers
- ✗ Poor for nonlinear data
- ✗ Overfitting possible

24. Real Project Flow

Load Dataset ↓

Cleaning ↓

Split ↓

Train ↓

Predict ↓

Evaluate ↓

Deploy

Short Summary

Linear Equation:

$$y = mx + c$$

Multiple:

$$y = b_0 + b_1x_1 + \dots$$

Normal Equation:

$$\beta = (X^T X)^{-1} X^T y$$

Metrics:

MAE

MSE

RMSE

R^2

'''